Big Data Processing Architectures and Their Role in the Future of AI Systems

In this essay, I want to take a technical perspective on how Big Data Processing Architectures will evolve with respect to current and future Artificial Intelligence workloads.

Analytical Processing Foundations

In this first part I want to emphasize the difference between analytical processing and traditional transactional processing.

Differences between analytical and transactional processing

This comparison is also often denoted as OLAP (online analytical processing) versus OLTP (online transactional processing).

Important to note is that neither type of data processing system is universally superior for any use-case. Often the question is how to make the best use of both types for your situation.

OLAP systems, such as data warehouses, are optimized for conducting complex data analysis for smarter decision-making by running queries at high speeds on large volumes of data. They are used by data scientists, business analysts, and knowledge workers.

On the contrary, OLTP is optimized for processing a massive number of transactions and used by frontline workers or customer self-service applications.

Technically, OLTP systems prioritize low-latency writes, strict consistency, and high concurrency. They typically use row-oriented storage and B+ tree indexes optimized for point lookups and short transactions. Additionally, they guarantee ACID properties, consisting of Atomicity, Consistency, Isolation, and Durability.

In contrast, OLAP systems prioritize scan efficiency, aggregation performance, and compression. They usually use columnar storage formats that enable vectorized execution and predicate pushdown. Instead of optimizing for individual row access, they optimize for large-scale aggregations across billions of records.

How indexes, materialized views, and query optimization support analytical workloads

There are many different ways to optimize query response time, not only in OLTP, but also in OLAP. I want to focus on analytical processing in this essay.

Indexing

One of the purposes of OLAP databases is to make large volumes of data easily queryable with minimal latencies. To decrease these latencies, indexing is often used. An index is typically a tree-based structure defined on one or more columns that allows the system to locate relevant rows without scanning the entire table.

Common indexes used by analytical systems are bitmap indexes, zone maps, and LSM-tree-based indexing structures for large-scale distributed storage.

Materialized Views

By storing the results of complex queries in so-called materialized views, you reduce the need to recompute the results each time. Contrary to normal views, materialized views are updated periodically.

More advanced systems support Incremental View Maintenance (IVM), where only the delta, meaning the changes in the base tables, are propagated to the materialized view. This becomes particularly important for AI workloads, where the need for accurate data and high query rates make full recomputation difficult.

Query Optimization

There are many ways to optimize query response time by simply rewriting the query by using appropriate join strategies, avoiding selecting unnecessary columns, etc.

Other methods to minimize latency

By dividing large tables into smaller, more manageable pieces, queries only have to scan the relevant partition, rather than the entire table. This is called **partitioning**.

Another way to provide fast responses can be achieved by using an **optimized infrastructure** for the OLAP system in terms of hardware resources.

Streaming, Event Processing, and CDC

In this section I analyze how stream processing systems, event streaming platforms, and Change Data Capture pipelines complement or replace batch-oriented analytics.

In batch processing, all the data is gathered and processed at set times, such as every day, week, or month. This approach is for instance commonly used in banking, where batch loads are used to process large volumes of transactions overnight. However, it also increases latency.

Event streaming platforms act as durable logs of ordered events. They decouple producers and consumers and enable replayability. This is a crucial property for reproducibility in AI pipelines.

Differences and overlaps between stream processing and incremental analytics

In stream processing, data is processed in chunks as it flows in. That works very well for log-heavy or monitoring tasks, but there are also some downsides to this approach. Dealing with state snapshots, failover, and out-of-order event handling requires many resources and demands constant monitoring, inflating costs and skill requirements beyond simpler batch setups. Furthermore, the concept of eventual consistency can lead to inconsistencies at window boundaries, where accurate results are wanted.

Incremental analytics like Change Data Capture (CDC) detect inserts, updates, or deletes from sources in real time and refresh only what is affected.

Where stream processing is event-reactive, focused on minimal lag and uses windowing with watermarks, incremental analytics is delta-driven and propagates changes without necessarily relying on time windows.

However, there is strong overlap between both paradigms. Modern stream processors, like Apache Flink and Spark Structured Streaming, effectively implement incremental computation over unbounded data streams.

From a theoretical perspective, stream processing can be viewed as continuous incremental view maintenance.

Together, they form the backbone of real-time analytical processing.

Trade-offs between Latency and Consistency

Batch-oriented analytics ensures consistency at the cost of high latency.

Stream Processing prioritizes low latency but often operates under eventual consistency models, especially in distributed environments.

Incremental analytics attempts to balance both by updating only affected data while maintaining correctness guarantees.

Implications for AI Systems

This part analyzes whether the discussed processing architectures support or constrain modern AI workloads like training LLMs, or AI agents that continuously generate data, feedback, and queries.

Data Processing in Training and Fine-tuning LLMs

With LLMs, continuously updated and accurate data is important. However, real-time updates are usually not required for the base model training.

Since training a Large Language Model is computationally expensive, models are typically trained once on a massive knowledge base and then fine-tuned for domain-specific tasks.

The so-called foundation models are typically trained in large offline batch jobs on static snapshots of curated datasets. This favors batch-oriented architectures with strong reproducibility guarantees.

However, fine-tuning, reinforcement learning from human feedback (RLHF), and continual learning introduce incremental data pipelines. Here, CDC and streaming systems become important for collecting user feedback, filtering evidence, and generating updated training datasets.

Retrieval-Augmented Generation (RAG)

RAG is a more cost-effective approach to introducing new data to the LLM, since retraining is computationally and financially expensive.

Outputs of LLMs are improved by referencing an authoritative knowledge base outside of the model's training data sources before the response is generated. This is done by creating a knowledge library of accessible data sources, which occurs most often as a vector database. Relevant information is retrieved by converting the user query to a vector representation and matching it with this database. The RAG model augments the user input by adding the relevant retrieved data in context.

It is very important to avoid letting this external data become stale. Asynchronous updates of the documents in the database and their embedding representation are necessary.

This introduces a new architectural layer, the so-called serving layer. It handles the query throughput, low-latency vector similarity search, continuous ingestion of updated documents, and re-embedding and re-

indexing strategies. Therefore, AI workloads amplify the importance of incremental indexing, as the cost of re-indexing the entire database would be prohibitive.

Both real-time processing and periodic batch processing are acceptable implementation strategies. In case the external data changes very frequently, real-time processing is preferable as updates are monitored and accomplished almost immediately. Additionally, not all the data is copied again, only the changes that had been made.

Agentic AI

Agentic AI builds on generative AI techniques by using LLMs to function in dynamic environments. Agentic AI models apply generative outputs towards specific goals without constant human oversight and directly interact with external tools or databases.

This introduces new architectural challenges: what works for simple AI assistants will no longer be sufficient for truly autonomous agents capable of dynamic decision-making and coordination across domains. Instead, architectures increasingly combine event logs, stream processors, analytical warehouses, and serving layers optimized for low-latency reads.

Technical Positioning and Future Outlook

In this final part I take a clear position on whether AI workloads will push data systems towards unified architectures or deeper specialization.

Push towards unified architectures

I expect AI to push towards architectures that implement real-time processing, consisting of CDC connectors between the data sources and stream processing logs. However, different decisions based on the use-case are still necessary, sometimes an architecture like that may perform worse than a batch-based processing approach.

Central Components

Optimizing the performance of the serving layer will be a central component of architectures for AI workloads. It is not only responsible for handling the high query throughput, but also for combining the different knowledge bases and returning the response.

Deeper specialization in terms of the serving layer is to be expected, it needs to deal with many different systems depending on the use-case.

Furthermore, also incremental computing, especially in terms of incremental view maintenance will play a big role in the future. AI workloads require massive amounts of data, which makes recomputation very expensive. Therefore, incremental analytics provides many advantages.

Architectural principles

Apart from incremental computation and log-centric architectures, scalability is also of great interest for AI workloads. Additionally, the trade-offs between availability and consistency will still play a huge role in future design of architectures.

Since availability is a vital property for chatbots, much research will be conducted on how to ensure accurate responses with up-to-date information while maintaining high availability for users.

References

IBM. "OLAP vs OLTP." https://www.ibm.com/think/topics/olap-vs-oltp. [visited 08.02.2026]

Medium. "Learning Multi-Dimensional Indices". https://medium.com/data-science/learning-multi-dimensional-indices-a7aaa2044d8e. [visited 08.02.2026]

TapData. "Stream Processing vs Incremental Computing". https://tapdata.io/blog/stream-processing-vs-incremental-computing. [visited 09.02.2026]

AWS. "What is Retrieval-Augmented Generation?". https://aws.amazon.com/what-is/retrieval-augmented-generation/. [visited 09.02.2026]

IBM. "What is agentic AI?". https://www.ibm.com/think/topics/agentic-ai. [visited 10.02.2026]

Dataversity. Varthakavi, Mohan. "Reimagining Data Architecture for Agentic AI". https://www.dataversity.net/articles/reimagining-data-architecture-for-agentic-ai/. [visited 10.02.2026]